

Assignment 4

Dual-core vector addition and timing

Matteo Tuzi
Student ID: 70/90/00490
Course: Advanced Embedded System

February 23, 2026

Contents

1	Purpose of the assignment	2
1.1	DDR memory map	3
2	Platform and tools	5
2.1	Build configuration (optimisation -O3)	5
2.2	Run configuration / System Debugger	5
3	Software architecture	6
3.1	Master role (Core 0)	6
3.2	Worker role (Core 1)	6
3.3	Shared memory	6
4	Timing measurement	8
4.1	UART output	8
4.2	Results for different sizes	8
5	Interpretation of results	11
5.1	Assembly verification and NEON auto-vectorisation	12
6	Conclusions	13

1 Purpose of the assignment

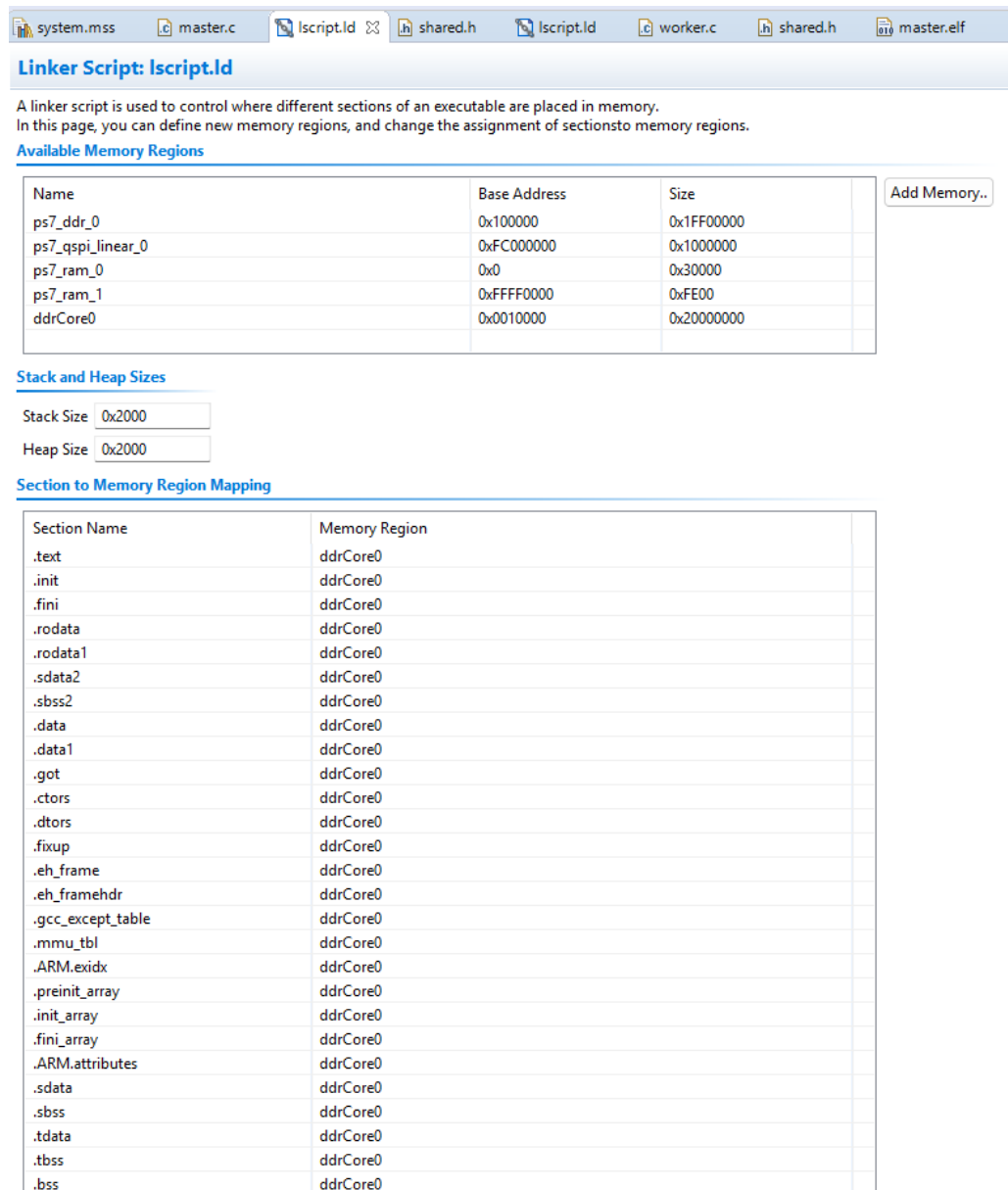
The assignment requires implementing a bare-metal application on Zynq in which the two ARM Cortex-A9 cores cooperate to compute a vector sum $C = A + B$. Two execution modes are compared:

- **Sequential:** a single core (Core 0) computes the entire vector C .
- **Parallel:** Core 0 computes the first half of C , Core 1 the second half; synchronisation is achieved via flags in shared memory.

The goal is to measure execution time with the ARM Global Timer and derive the speedup.

1.1 DDR memory map

The following figures show the DDR region layout for the master (Core 0) and the worker (Core 1), as configured in the linker scripts.



Linker Script: Iscript.ld

A linker script is used to control where different sections of an executable are placed in memory. In this page, you can define new memory regions, and change the assignment of sections to memory regions.

Available Memory Regions

Name	Base Address	Size
ps7_ddr_0	0x100000	0x1FF00000
ps7_qspi_linear_0	0xFC000000	0x1000000
ps7_ram_0	0x0	0x30000
ps7_ram_1	0xFFFF0000	0xFE00
ddrCore0	0x0010000	0x20000000

Stack and Heap Sizes

Stack Size: 0x2000

Heap Size: 0x2000

Section to Memory Region Mapping

Section Name	Memory Region
.text	ddrCore0
.init	ddrCore0
.fini	ddrCore0
.rodata	ddrCore0
.rodata1	ddrCore0
.sdata2	ddrCore0
.sbss2	ddrCore0
.data	ddrCore0
.data1	ddrCore0
.got	ddrCore0
.ctors	ddrCore0
.dtors	ddrCore0
.fixup	ddrCore0
.eh_frame	ddrCore0
.eh_framehdr	ddrCore0
.gcc_except_table	ddrCore0
.mmu_tbl	ddrCore0
.ARM.exidx	ddrCore0
.preinit_array	ddrCore0
.init_array	ddrCore0
.fini_array	ddrCore0
.ARM.attributes	ddrCore0
.sdata	ddrCore0
.sbss	ddrCore0
.tdata	ddrCore0
.tbss	ddrCore0
.bss	ddrCore0

Figure 1: DDR memory Core0 (master).

system.mss
master.c
lscript.ld
shared.h
lscript.ld
worker.c
shared.h
master.elf

Linker Script: lscript.ld

A linker script is used to control where different sections of an executable are placed in memory. In this page, you can define new memory regions, and change the assignment of sections to memory regions.

Available Memory Regions

Name	Base Address	Size	
ps7_ddr_0	0x100000	0x1FF00000	
ps7_qspi_linear_0	0xFC000000	0x1000000	
ps7_ram_0	0x0	0x30000	
ps7_ram_1	0xFFFF0000	0xFE00	
ddrCore1	0x030000000	0xFF00000	

Add Memory..

Stack and Heap Sizes

Stack Size
0x2000

Heap Size
0x2000

Section to Memory Region Mapping

Section Name	Memory Region
.text	ddrCore1
.init	ddrCore1
.fini	ddrCore1
.rodata	ddrCore1
.rodata1	ddrCore1
.sdata2	ddrCore1
.sbss2	ddrCore1
.data	ddrCore1
.data1	ddrCore1
.got	ddrCore1
.ctors	ddrCore1
.dtors	ddrCore1
.fixup	ddrCore1
.eh_frame	ddrCore1
.eh_framehdr	ddrCore1
.gcc_except_table	ddrCore1
.mmu_tbl	ddrCore1
.ARM.exidx	ddrCore1
.preinit_array	ddrCore1
.init_array	ddrCore1
.fini_array	ddrCore1
.ARM.attributes	ddrCore1
.sdata	ddrCore1
.sbss	ddrCore1
.tdata	ddrCore1
.tbss	ddrCore1
.bss	ddrCore1

Figure 2: DDR memory Core1 (worker).

2 Platform and tools

- **Board:** Zybo Z7 (Zynq-7000, dual ARM Cortex-A9).
- **Environment:** Xilinx SDK or Vitis; serial terminal 115200 8N1.
- **Peripherals:** Shared DDR, ARM Global Timer, AXI GPIO (button), UART.

The two cores use separate linker scripts: the master uses one DDR region (e.g. ddrCore0), the worker another (e.g. ddrCore1). The shared memory region (e.g. from 0x2F100000) must not be assigned to any section by the linkers.

2.1 Build configuration (optimisation -O3)

The **master** and **worker** applications were built with **optimisation -O3** (Optimize most). In Xilinx SDK/Vitis: **Project** → **Properties** → **C/C++ Build** → **Settings** → **Optimization**, set level -O3 for both projects.

With -O3 the compiler enables aggressive optimisations and vectorisation (`-ftree-vectorize`, `-mvectorize-with-neon-quad`) applies to the serial loop; the worker also runs its half of the vector efficiently, reducing the time the master waits in `done1`. A Debug (-O0) build would yield much higher parallel cycles and an artificially low speedup.

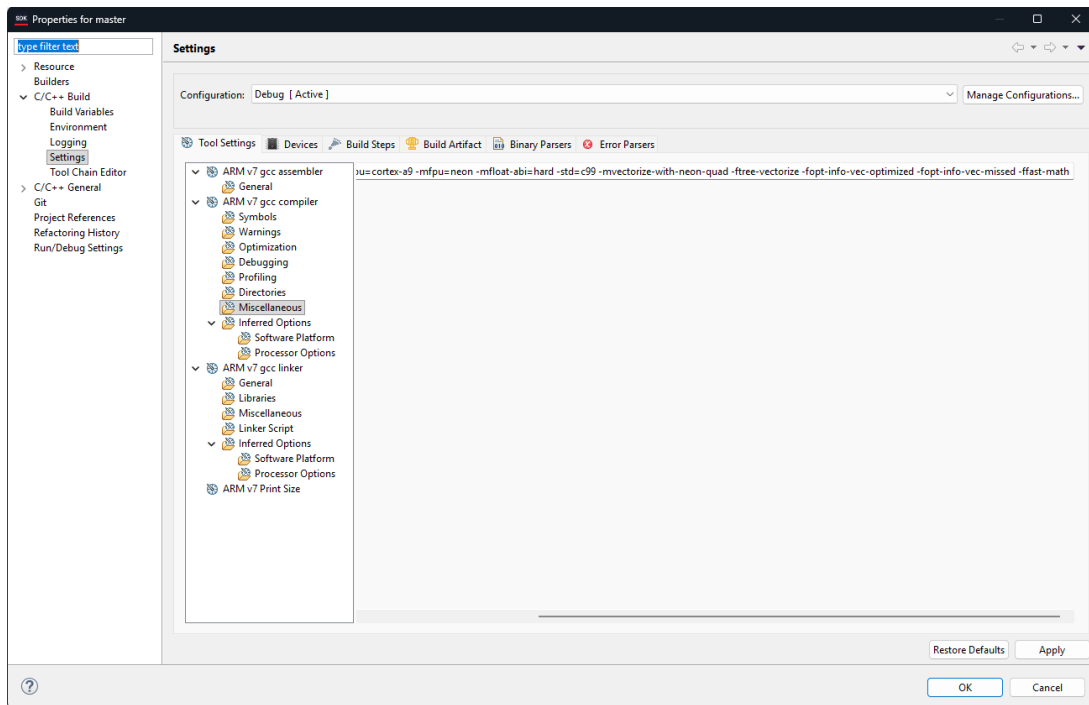


Figure 3: Compiler options for master project (-O3 and NEON flags).

2.2 Run configuration / System Debugger

The debug session uses a **System Debugger** configuration: `master.elf` is loaded on Core 0 (ps7_cortexa9_0) and `worker.elf` on Core 1 (ps7_cortexa9_1), with FPGA programming and system reset enabled, so that both cores start correctly.

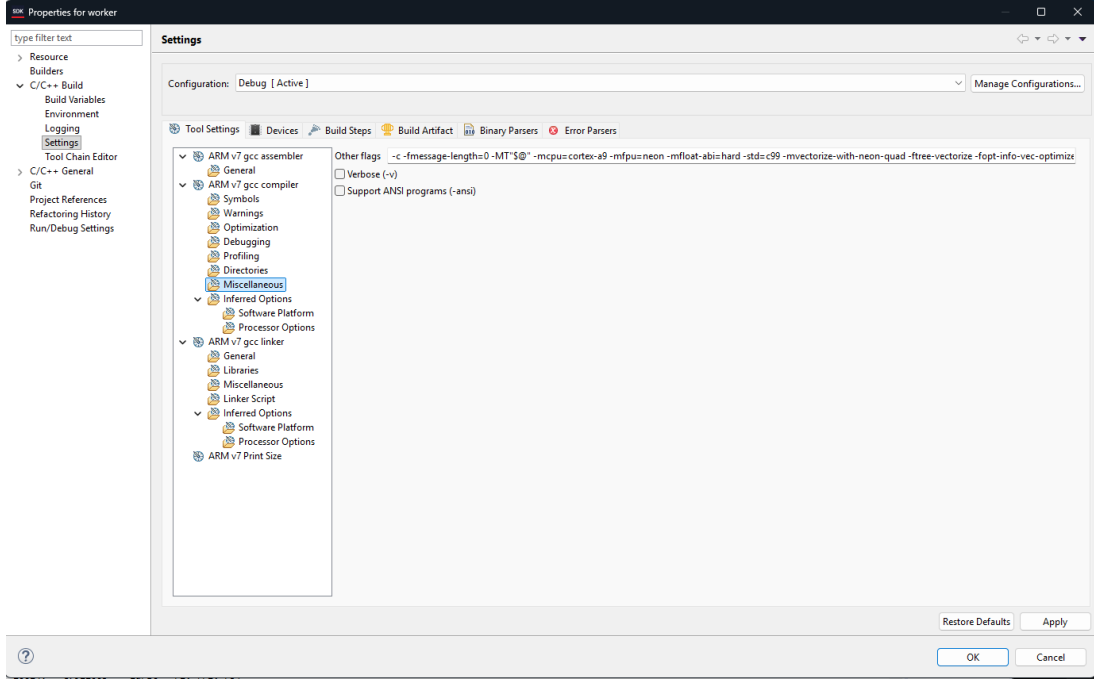


Figure 4: Compiler options for worker project (-O3 and NEON flags).

3 Software architecture

3.1 Master role (Core 0)

The master disables caches, waits for the GPIO button, initialises vectors A , B and flags in shared memory. It runs the vector add in serial mode and measures its cycles (baseline). Then it clears C , waits for `ready1`, sets `start`, computes the first half of C , waits for `done1`, measures the parallel-phase cycles, computes the speedup and checks that C matches the serial result.

3.2 Worker role (Core 1)

The worker disables caches, initialises `ready1` and `done1`, immediately signals `ready1 = 1`, waits for `start`, computes the second half of C , sets `done1 = 1` and enters idle.

3.3 Shared memory

The structure in `shared.h` contains vectors A , B , C and flags `ready1`, `start`, `done0`, `done1`. The base address is defined by `SHARED_BASE` (e.g. `0x2F100000U`).

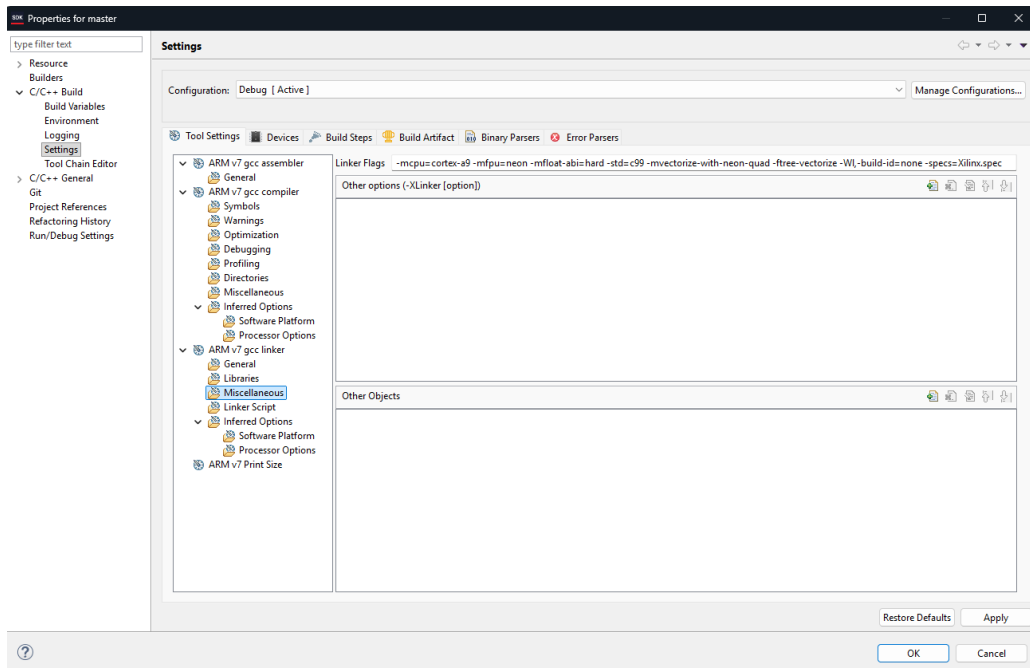


Figure 5: Master linker script: section placement in DDR Core0.

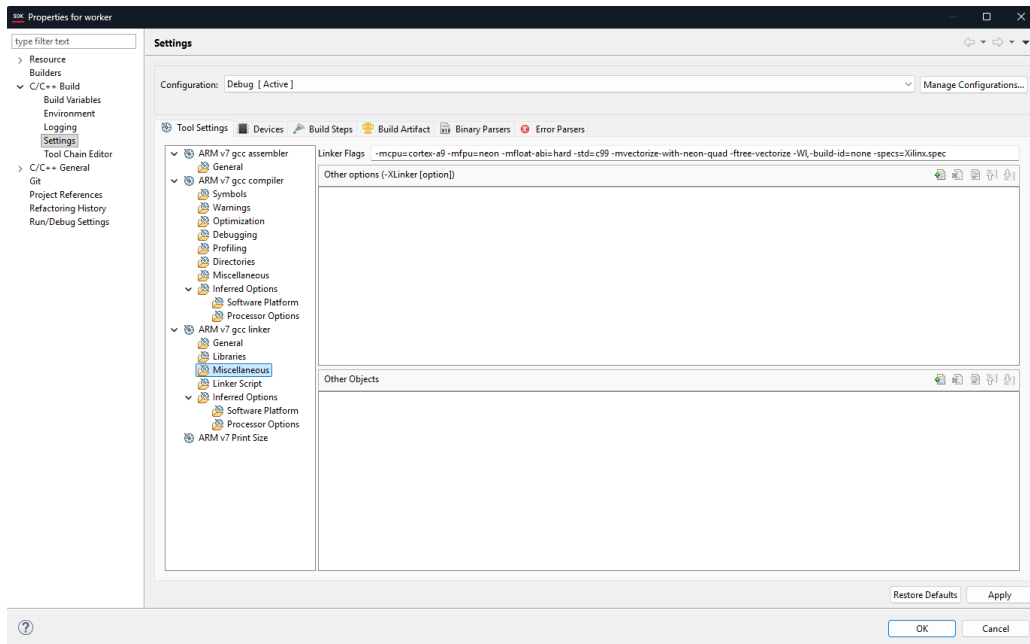


Figure 6: Worker linker script: section placement in DDR Core1.

4 Timing measurement

The Global Timer (32-bit register, `XPAR_GLOBAL_TMR_BASEADDR + 0x00`) is used. For each phase the timer is read immediately before and after the block under measurement; the difference is the cycle count. Caches are disabled for deterministic timing. Results are expressed in cycles; the timer frequency is approximately 333 MHz (platform-dependent).

4.1 UART output

The master output on the serial terminal (115200 8N1) includes: boot message with `N`, serial and parallel run cycles, speedup and number of validation errors (zero if the parallel result matches the serial one). Figure 7 shows the example for `ARRAY_SIZE=10`.

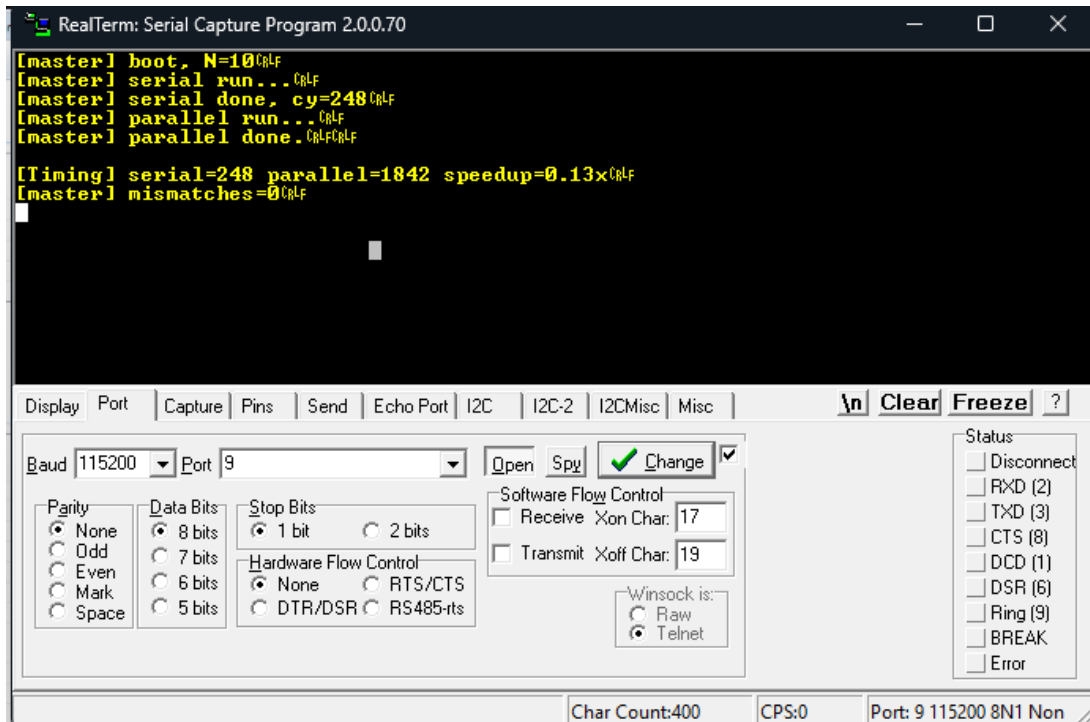


Figure 7: UART output for `ARRAY_SIZE=10`.

4.2 Results for different sizes

Tests were run with `ARRAY_SIZE = 10, 100, 1000` and `5000`. Below are the UART output screenshots for each size (sequential placement).

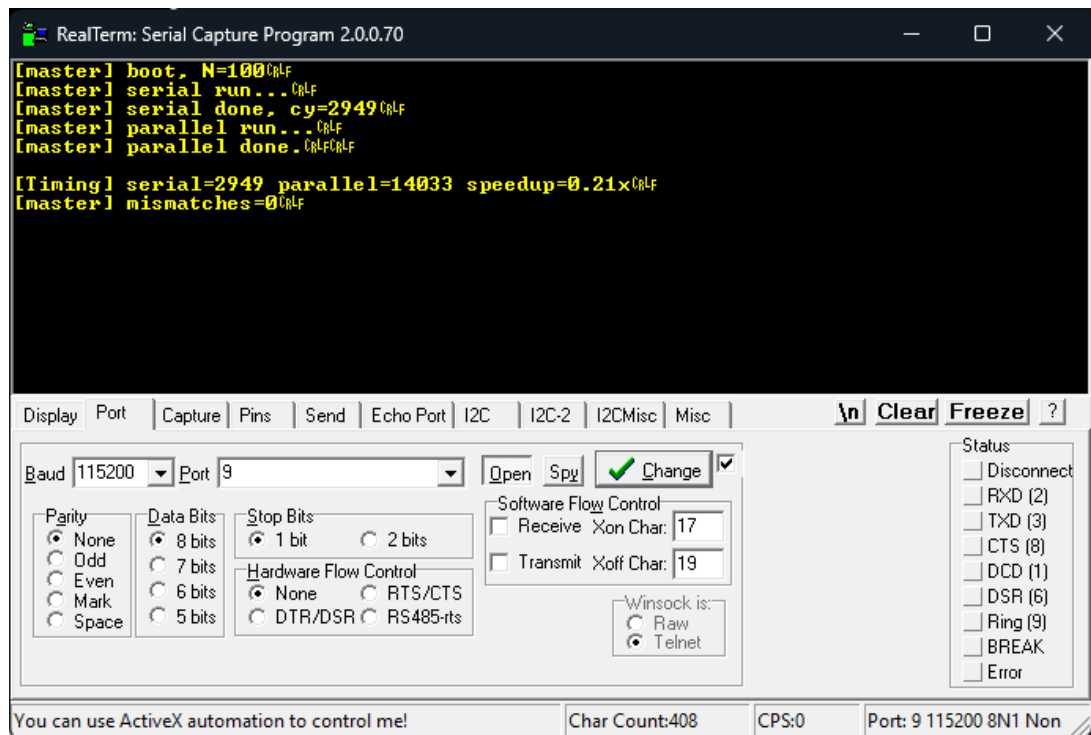


Figure 8: UART output for ARRAY_SIZE=100.

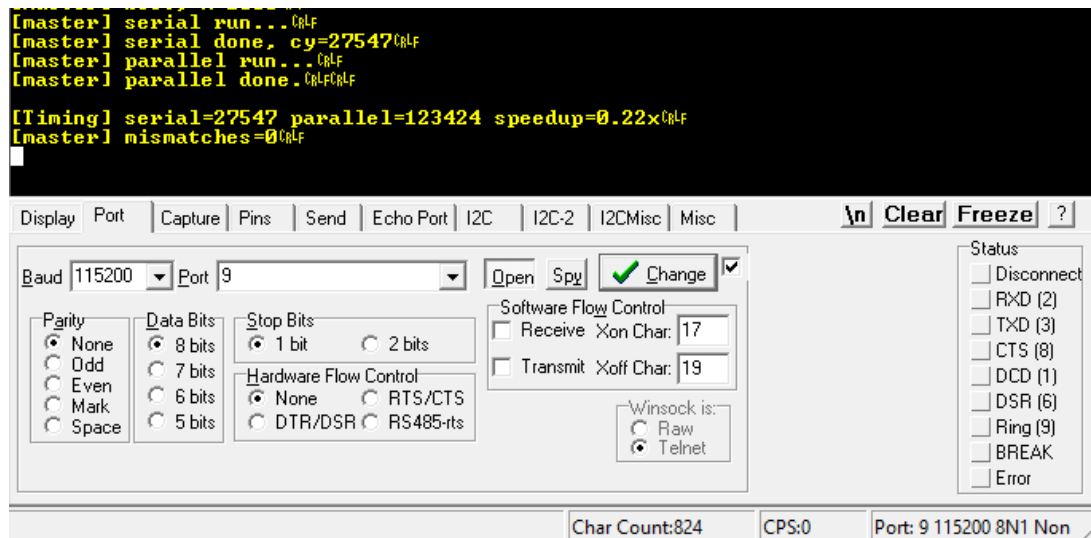


Figure 9: UART output for ARRAY_SIZE=1000.

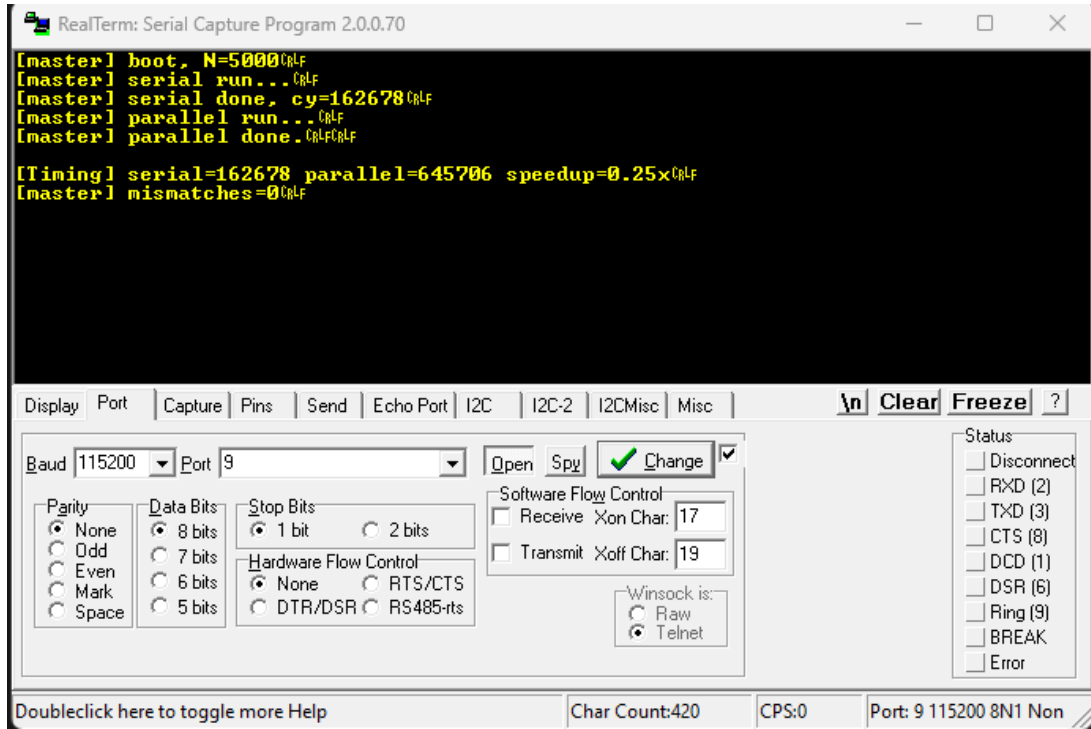


Figure 10: UART output for `ARRAY_SIZE=5000`.

Summary table. Values read from the screenshots for the four sizes tested.

ARRAY_SIZE	Serial cycles	Parallel cycles	Speedup
10	248	1 842	0.13
100	2 949	14 033	0.21
1000	27 547	123 424	0.22
5000	162 678	645 706	0.25

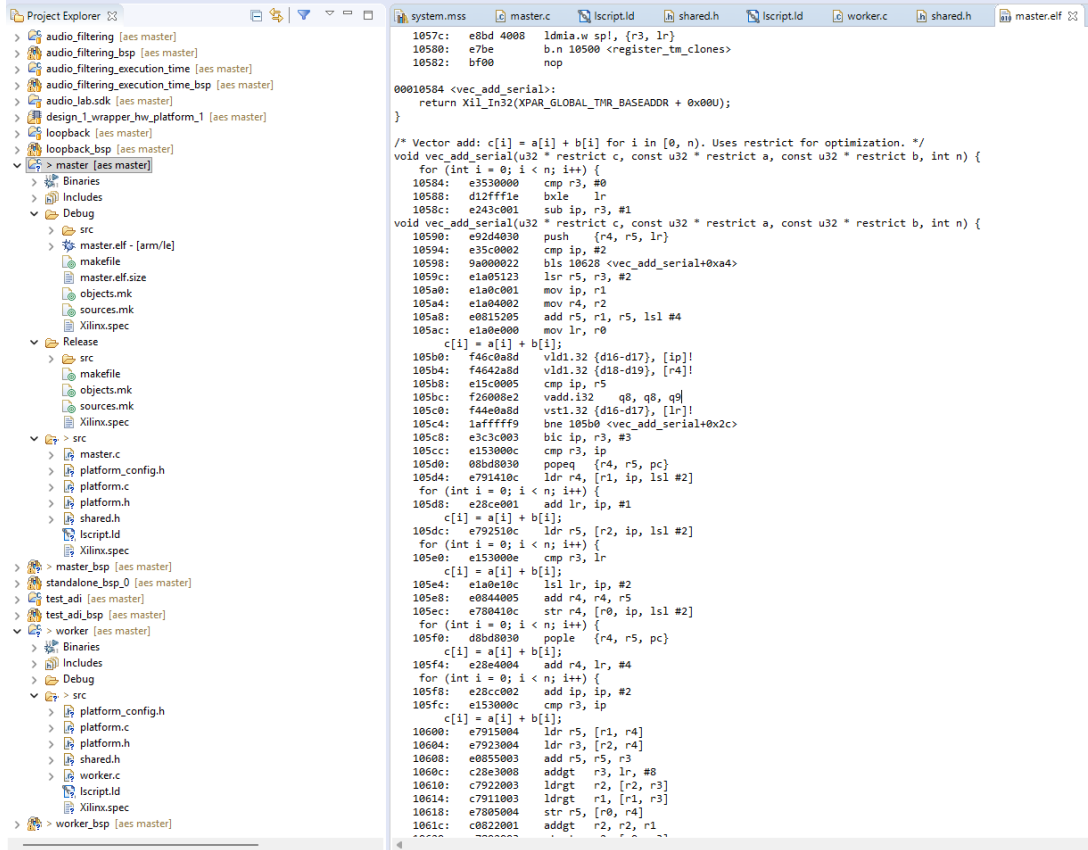
5 Interpretation of results

Results for `ARRAY_SIZE` = 10, 100, 1000 and 5000 show that parallel execution (two cores) does not achieve a speedup close to 2x over the serial baseline. In many cases the parallel-phase cycles are comparable to or higher than the serial ones, so the speedup is ≤ 1 .

This behaviour is expected on an architecture with **shared DDR memory** and **disabled caches**: the vector add kernel is **memory-bound**, i.e. execution time is dominated by memory accesses rather than computation. Both cores access the same DDR, causing **bandwidth contention**; the benefit of having two cores working in parallel is therefore limited by available bandwidth. With small `ARRAY_SIZE` (10, 100) the **synchronisation overhead** (waiting for `ready1`, `start`, `done1`) can be comparable to compute time, further worsening the serial/parallel ratio. As vector size increases, useful work grows but contention on shared memory remains the limiting factor.

5.1 Assembly verification and NEON auto-vectorisation

The serial code (`vec_add_serial`) was compiled with `-O3` optimisation and NEON vectorisation flags. Inspection of the `master.elf` disassembly confirms the presence of **ARM NEON instructions** in the loop: `vld1.32` (vector load), `vadd.i32` (32-bit integer vector add), `vst1.32` (vector store). The compiler therefore applied auto-vectorisation to the serial loop, exploiting data-level parallelism (SIMD); this reduces the number of instructions executed per element but does not remove the shared-memory bottleneck in the parallel phase.



The screenshot shows an IDE with a project explorer on the left and a disassembly window on the right. The project explorer shows a project named 'master' with various source files. The disassembly window shows the assembly code for the `vec_add_serial` function. The code is written in ARM assembly, with comments in C. The function is defined as `void vec_add_serial(u32 * restrict a, const u32 * restrict b, int n) {`. The assembly code shows the function's implementation, including a loop that iterates over the input data. The code is optimized for NEON, using vector instructions like `vld1.32`, `vadd.i32`, and `vst1.32`. The disassembly window also shows the function's return value and the end of the function.

```
1057c: e8bd 4008 ldmia.w sp!, {r3, lr}
10580: e7be      b.n 10500 <register_tm_clones>
10582: bf00      nop

00010584 <vec_add_serial>:
return Xil_In32(XPAR_GLOBAL_THR_BASEADDR + 0x000);
}

/* Vector add: c[i] = a[i] + b[i] for i in [0, n). Uses restrict for optimization. */
void vec_add_serial(u32 * restrict a, const u32 * restrict b, int n) {
    for (int i = 0; i < n; i++) {
10584: e3530000 cmp r3, #0
10588: d12ffff1e bxle lr
1058c: e243c001 sub ip, r3, #1
10590: e92d4030 push {r4, r5, lr}
10594: e35c0002 cmp ip, #2
10598: 9a000022 bls 10620 <vec_add_serial+0xa4>
1059c: e1a05123 lsr r5, r3, #2
105a0: e1a0c001 mov ip, r1
105a4: e1a04002 mov r4, r2
105a8: e0815205 add r5, r1, r5, lsl #4
105ac: e1a0e000 mov lr, r0
105b0: f46c0a8d vld1.32 {d16-d17}, [ip]!
105b4: f4642a8d vld1.32 {d18-d19}, [r4]!
105b8: e15c0005 cmp ip, r5
105bc: f26000e2 vadd.i32 q8, q8, q8
105c0: f44e0a8d vst1.32 {d16-d17}, [lr]!
105c4: 1affffff bne 105b0 <vec_add_serial+0x2c>
105c8: e3c3c003 bic ip, r3, #3
105cc: e153000c cmp r3, ip
105d0: 08bd8030 popeq {r4, r5, pc}
105d4: e791410c ldr r4, [r1, ip, lsl #2]
105d8: e28ce001 add lr, ip, #1
105dc: e792510c ldr r5, [r2, ip, lsl #2]
105e0: e153000e cmp r3, lr
105e4: e1a0e10c lsl lr, ip, #2
105e8: e0844005 add r4, r4, r5
105ec: e780410c str r4, [r0, ip, lsl #2]
105f0: d8bd8030 pople {r4, r5, pc}
105f4: e28e4804 add r4, lr, #4
105f8: e28cc002 add ip, ip, #2
105fc: e153000c cmp r3, ip
10600: e7915004 ldr r5, [r1, r4]
10604: e7923004 ldr r3, [r2, r4]
10608: e0855003 add r5, r5, r3
1060c: c28e3008 addgt r3, lr, #8
10610: c7922003 ldrgt r2, [r2, r3]
10614: c7911003 ldrgt r1, [r1, r3]
10618: e7805004 str r5, [r0, r4]
1061c: c0822001 addgt r2, r2, r1
```

Figure 11: Disassembly of the serial function with NEON instructions (`vld1.32`, `vadd.i32`, `vst1.32`).

6 Conclusions

The assignment led to implementing a bare-metal dual-core application on Zynq with coordination via shared memory and software flags. Serial and parallel execution of the vector add were compared, measuring cycles with the ARM Global Timer and verifying that the parallel result matches the serial one (zero errors).

The results confirm that, without caches and with shared DDR, a memory-bound kernel such as vector add does not benefit significantly from two-core parallelism: speedup remains limited or below 1 due to bandwidth contention. Building with -O3 and using NEON improves serial performance, but the main limit remains the memory architecture. The experiment is therefore consistent with the expected behaviour and with the literature on memory-bound workloads on embedded multi-core systems.